

How FIDO2 works, a technical deep dive – Michael Waterman



michaelwaterman.nl/2025/04/02/how-fido2-works-a-technical-deep-dive

Michael Waterman

April 2, 2025

Earlier this year, I completed my bachelor thesis in Cybersecurity, diving into one of the most exciting developments in the world of authentication (*well, that's my personal opinion anyways*), FIDO2 and Passkeys. My research focused on how passwordless authentication can reshape the way we secure digital identities, not only from a technical point of view, but also in terms of user experience, adoption, and the shift in mental models required to move beyond passwords.

This blog post builds on that research and aims to provide a technical deep dive into how FIDO2 actually works under the hood. While the promise of “passwordless and phishing-resistant” sounds great on paper, I wanted to truly understand, and now explain, what makes FIDO2 and Passkeys secure, very user friendly, and future-proof.

We'll start with a look at the cryptographic foundation that powers FIDO2, asymmetric encryption. From there, we'll walk through the two core flows, registration and authentication, and show how these interactions keep your credentials safe, even if an attacker is trying their best to trick you. One of those very common tricks is using a toolkit named Evilginx, a fairly easy to use tool to steal all your credentials, even when you're using MFA with number matching. Want to see it in action? Scroll down below and see the evil magic.

If you've ever wondered what happens behind the scenes when you log in with a Passkey, or why FIDO2 is being adopted by tech giants like Apple, Google, and Microsoft, this post is for you.

Asymmetric encryption

At the heart of FIDO2 lies asymmetric cryptography, a system that uses two mathematically linked keys: a private key and a public key.

- The **private key** is securely stored on your device and never leaves it.
- The **public key** is shared with the online service during registration.

A simple analogy: Lock and the key

Imagine you have a special kind of lock and a matching key. But this lock is a little unusual, you can only lock it with one key (the private key), and only unlock it with the other (the

public key). You keep the private key hidden in your pocket, but you hand out the public key to anyone who wants to check if a message really came from you.

Now, if you lock a message using your private key (by signing it), anyone can try to unlock it using your public key. If it works, they know two things:

1. The message hasn't been changed.
2. It really came from you, because only your private key could have created that valid "lock."

That's essentially how FIDO2 works. You sign something with your private key, and the server verifies it using your public key. No one else can fake your signature unless they have your private key and since that key never leaves your device, it's extremely hard to steal.

Nothing new

Asymmetric encryption isn't new. In fact, it has been a cornerstone of digital security since the late 1970s, when RSA (Rivest-Shamir-Adleman) was introduced. The principles behind RSA, large prime number factorization and [modular](#) arithmetic, still underpin much of our secure internet infrastructure today.

Protocols like TLS (for HTTPS), PGP (for email encryption), and SSH (for secure remote login) all rely on public-private key cryptography. What FIDO2 does differently is not inventing a new cryptographic system, but rather applying existing, proven technology in a way that improves user experience and eliminates shared secrets, like passwords.

Demo Time! Digital signature verification

To show how this works in practice, here's a simplified version of the cryptographic validation process that happens during a FIDO2 authentication request.

This PowerShell script does three things:

1. Loads a public key.
2. Loads a signed message and its signature.
3. Verifies the signature using SHA-256 hashing and PKCS#1 v1.5 padding.

```
# Create a new RSA Key Pair (2048-bit)
$rsa = [System.Security.Cryptography.RSACryptoServiceProvider]::new(2048)

# Export the private and public keys (optional as XML)
$privateKey = $rsa.ToXmlString($true)
$publicKey = $rsa.ToXmlString($false)

# Example data
```

```

$data = "FIDO2 is the Future"
$bytes = [System.Text.Encoding]::UTF8.GetBytes($data)

# Sign the data (SHA256 hash + RSA)
$signature = $rsa.SignData($bytes,
[System.Security.Cryptography.HashAlgorithmName]::SHA256,
[System.Security.Cryptography.RSASignaturePadding]::Pkcs1)

# New RSA instance with the public key
$rsaVerify = [System.Security.Cryptography.RSACryptoServiceProvider]::new()
$rsaVerify.FromXmlString($publicKey)

# Verify the signature
$isValid = $rsaVerify.VerifyData($bytes, "SHA256", $signature)

# Results
if ($isValid) {
    Write-Host "The signature is valid and is signed with the correct private key."
    -ForegroundColor Green
} else {
    Write-Host "The signature is not valid!" -ForegroundColor Red
}

```

Padding? Say what now?

You might wonder, what is PKCS#1 padding, and why does it matter?

When encrypting or signing data using RSA, we don't just apply the private key directly to the raw bytes. Instead, the data is first padded using a specific scheme to ensure:

- Consistent and safe structure
- Protection against certain cryptographic attacks (like chosen plaintext attacks)
- Compatibility with expected input sizes

Think of padding like placing a message into a standard-sized envelope, no matter how short the content inside, the outer format always looks the same. This prevents attackers from guessing or manipulating the message structure.

FIDO2 implementations commonly use RSASSA-PKCS1-v1_5 or RSASSA-PSS padding for signature operations. In our demo, we used Pkcs1, which is widely supported and standard for many applications, including legacy systems. Newer systems may prefer PSS, which offers slightly stronger theoretical security, but PKCS#1 is still considered safe and effective.

Technical note on ECDSA and Quantum resistance?

While the PowerShell demo above uses RSA, many FIDO2 authenticators in the real world use ECDSA (*Elliptic Curve Digital Signature Algorithm*), especially on mobile devices and security keys. ECDSA provides the same cryptographic principles as RSA but with:

- Smaller key sizes
- Faster performance
- Lower computational and memory overhead

For example, a 256-bit ECDSA key offers roughly the same security as a 3072-bit RSA key.

However, ECDSA is not quantum-resistant. Just like RSA, it's vulnerable to [Shor's algorithm](#), which could break it if large-scale quantum computers become viable. This has led to increasing interest in post-quantum cryptography, including new algorithms like Dilithium and Falcon, which are under review by NIST and FIDO Alliance for potential future use.

FIDO2 does allow for pluggable algorithms, so in the future, post-quantum Passkeys may become a reality, but today's deployments are still based on classical cryptography.

FIDO2 flow explained

Before we dive into the flow itself, it's important to understand that FIDO2 is built on two core protocols:

WebAuthn vs. CTAP – Who does what?

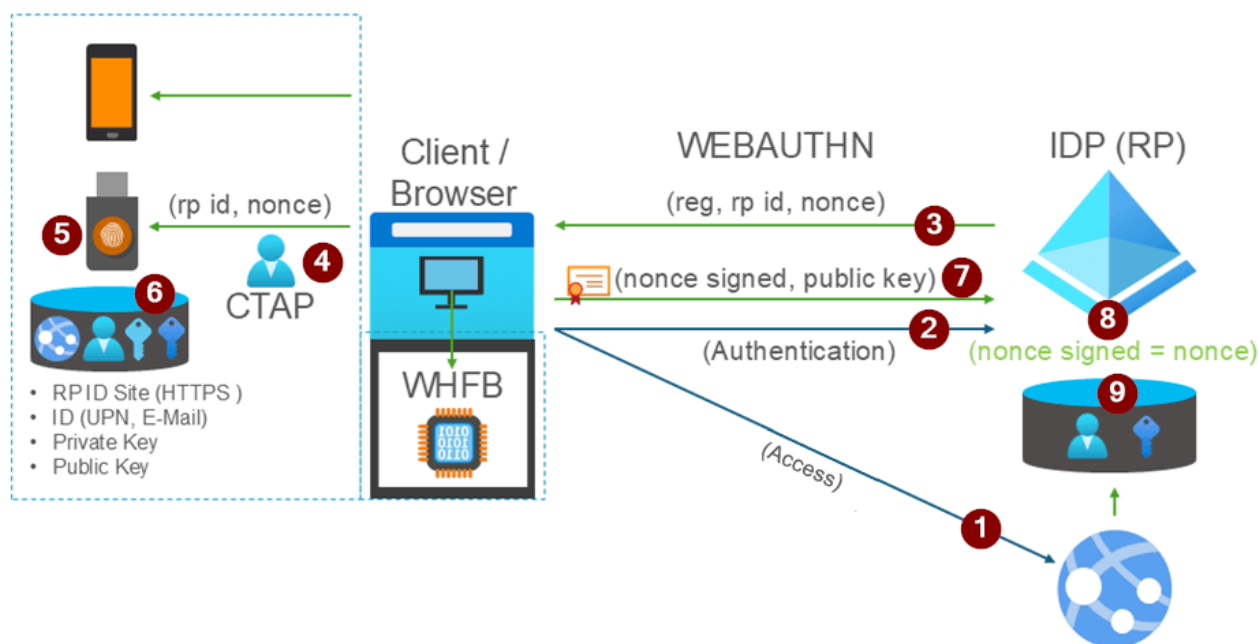
- *WebAuthn (Web Authentication API)* This is the browser-side protocol defined by the W3C. It allows web applications to trigger authentication or registration events via JavaScript. WebAuthn handles the communication between the browser (or OS) and the web service, sending challenges, receiving responses, and passing data securely.
- *CTAP (Client-to-Authenticator Protocol)* This is the device-side protocol developed by the FIDO Alliance. It defines how the browser or OS talks to the authenticator device, like a security key, smartphone, or built-in module like Windows Hello.

Think of it this way:

- WebAuthn = **from the browser to the website**
- CTAP = **from the browser to the authenticator**

Together, they form the FIDO2 standard. Whether you're using a USB key or biometric unlock, these protocols ensure secure, phishing-resistant communication from your finger tips to the cloud and yes I mean this quite literally.

So how does a Passkey actually get created and registered? Let's break down the FIDO2 registration process step by step, using the image below as a reference:



Step-by-step walkthrough

1. User navigates to a website

The journey begins when a user opens a FIDO2-enabled website, for example, *office.com*. The browser initiates a secure HTTPS connection and sends a request to log in.

2. Redirect to Identity Provider (IDP)

The user is redirected to the website's Identity Provider (IDP) or Relying Party (RP), this is the party that will manage the user's identity. For example *Entra ID (login.microsoftonline.com)* in the case of Microsoft.

3. WebAuthn challenge is issued

The RP sends a WebAuthn registration request to the browser. This contains three important elements:

- A Relying Party ID (RP ID): this is the website's domain, and it is **hashed** before being stored by the authenticator to preserve privacy.
- A *user identifier*: (e.g. email or UPN).
- A *nonce*: a one-time cryptographic challenge to prevent replay attacks.

4. User connects their authenticator

The user interacts with their authenticator — this could be a USB key, mobile phone, or a built-in authenticator like a TPM chip used by Windows Hello, all using the CTAP protocol.

5. Second factor and proximity verification

The authenticator verifies that the user is physically present through a local action: PIN entry, biometric scan, or device unlock.

6. Key pair generation and secure storage

The authenticator generates a new asymmetric key pair that is unique for the hashed RP ID. It securely stores:

The hashed RP ID (domain), The user ID (e.g. email address), The private key, and a credential ID (a reference used by the server to identify the public key later).

The corresponding public key will be sent to the RP.

7. Response to the RP

The browser sends a registration response containing:

- The public key, and
- The signed nonce, proving possession of the private key.

8. Verification by the RP

The RP verifies the signature using the received public key. If valid, it proves:

- The client controls the private key,
- The message is fresh and untampered,
- And the registration is legitimate.

9. Credential is stored

The RP stores the public key and user ID for future logins. This completes the registration step.

Authentication, same dance, different steps

Once a user is registered, logging in with a Passkey follows the same WebAuthn and CTAP flow, with a few key differences:

- No new key pair is generated. Instead, the authenticator looks up the stored key pair based on the hashed RP ID and credential ID.
- A new challenge (nonce) is sent by the RP, just like during registration.
- The private key is used to sign this challenge, proving that the user still possesses the correct authenticator and has performed a user verification step (like entering a PIN or touching a sensor).
- The signature and credential ID are returned to the RP, which then uses the previously stored public key to verify the signature.

This proves the user is who they claim to be, without requiring a password, and without revealing the private key at any point.

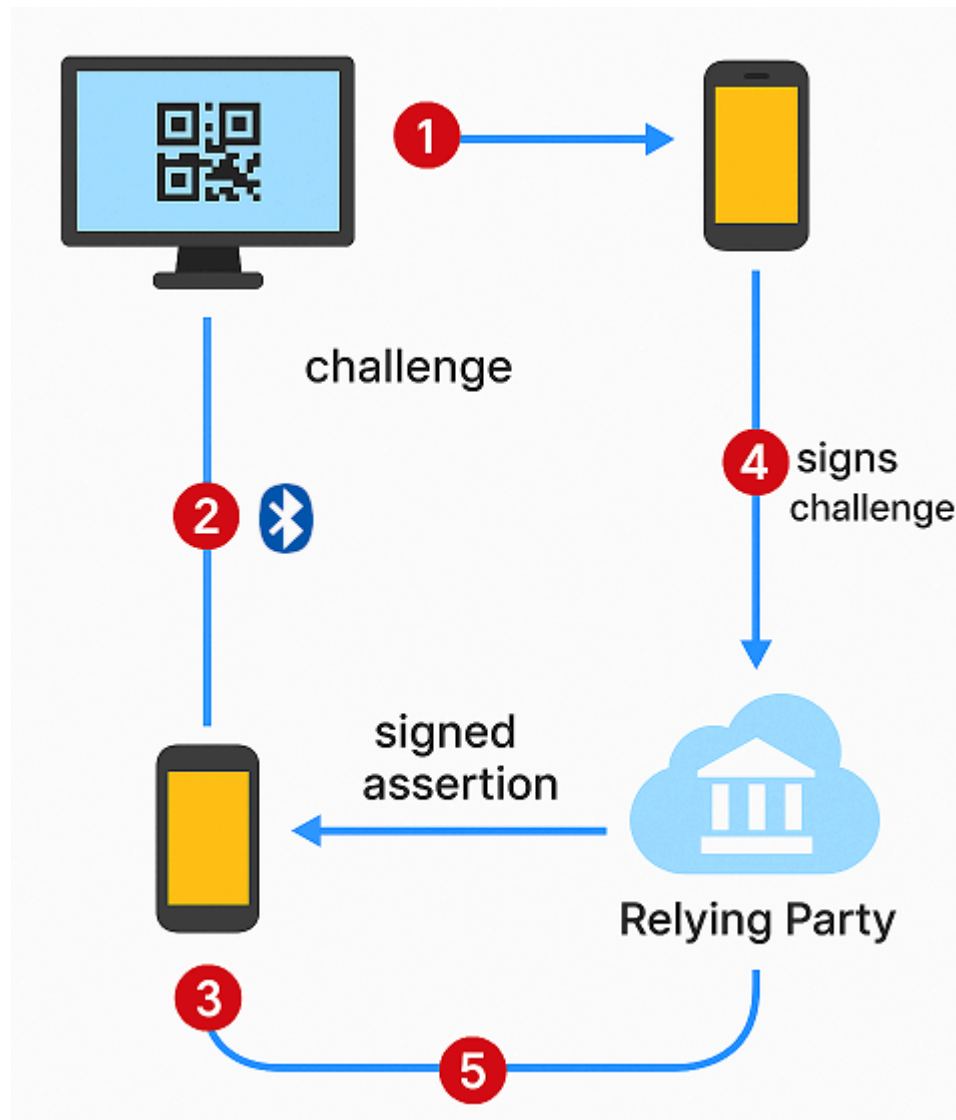
In essence, the authentication flow reuses the same trust model established during registration, creating a user friendly and phishing-resistant login experience.

Logging in with a QR code (Cross-Device Authentication)

One of the most user-friendly features of passkeys is the ability to log in on a device that doesn't have your Passkey, by using a device that does, like your phone. This is where QR

code-based login comes into play.

Let's say you want to log into a website on a desktop computer, but your Passkey is stored on your mobile phone. Instead of typing a password or syncing credentials, the website displays a QR code. Here's what happens behind the scenes:



1. You scan the QR code with your phone, which opens a deep link or browser session tied to that login request.

2. The QR code contains either the full login challenge (a cryptographic nonce) or a reference to an active login session on the relying party (RP) server. After scanning, your phone opens a secure WebAuthn flow, either in your browser or via the OS (like iCloud Keychain or Android). In many implementations, your phone also establishes a short-range Bluetooth Low Energy (BLE) connection with the desktop device. This local link:
 - Ensures physical proximity between the devices, strengthening phishing resistance
 - Helps bind the mobile response to the correct login session
 - May carry the encrypted assertion response back to the desktop, depending on the architecture

This local communication acts as a secure bridge for authentication, keeping the private key on your phone while allowing your desktop session to complete login.

3. The phone retrieves the challenge and metadata associated with the login request. This data is tied to the relying party's domain and protected by the same origin policy.
4. Your phone, which holds the passkey, signs the challenge using the private key associated with the RP ID.
 - It verifies that the RP domain matches the one the passkey is bound to.
 - It performs local user verification (PIN or biometric).
 - It signs the original challenge (nonce) that was issued at the start of the login session — the same one that would have been sent to a local authenticator.
 - It sends the signed assertion, along with the credential ID, back to the RP.
5. The relying party verifies the signature using the public key stored during registration, and if valid, the user is logged in, not on the phone, but on the desktop session that initiated the login.

This entire process is known as cross-device authentication. It still fully adheres to the FIDO2 principles:

- **No private keys are ever transferred** between devices.
- **Phishing-resistant** thanks to domain-bound credentials.
- **End-to-end encrypted communication** between the RP, the desktop, and the mobile authenticator.

The magic here is that the desktop device never gets access to your private key, it simply benefits from the trust established on your phone.

This feature makes passkeys truly practical in real-world scenarios, where users frequently move between devices and it shows how secure login doesn't have to mean friction.

Try it, webauthn.io

Curious how this works in practice, or whether your device even supports FIDO2? You can test it right now using webauthn.io, a free demo site maintained by Duo Security.

The site allows you to:

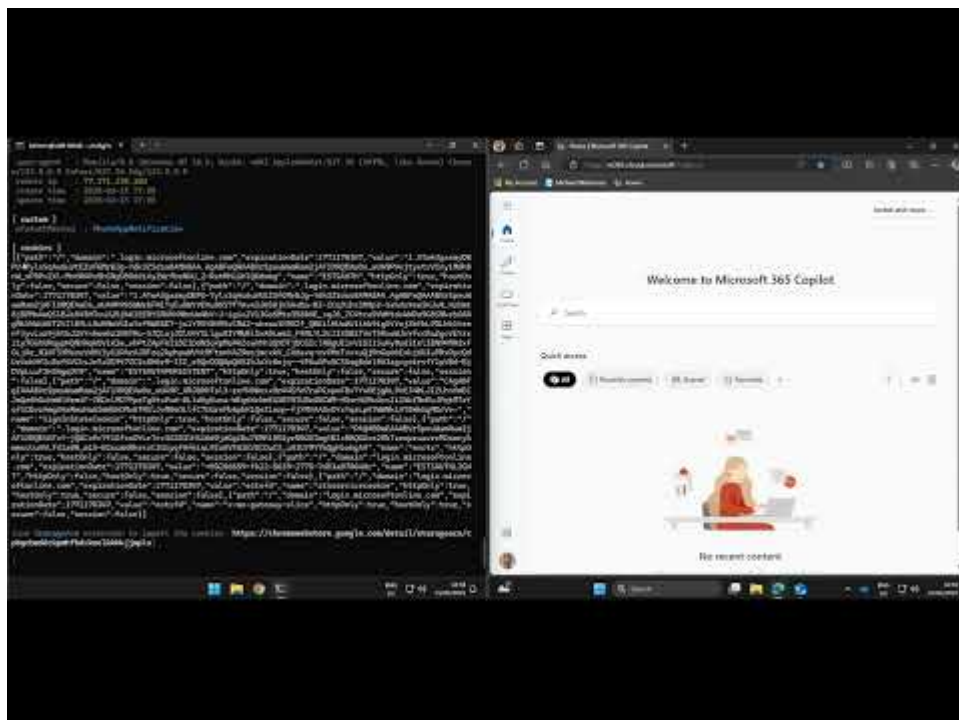
- Create a Passkey (a FIDO2 credential) in your browser
- Register a device authenticator, like a built-in fingerprint scanner or a USB/NFC security key
- Perform authentication using your newly created credential

It's a safe and effective way to understand the full flow of WebAuthn and CTAP, from registration to authentication, without needing to write a single line of code.

Pro tip: If you're using Chrome, Edge, or Safari on a modern device with biometrics, chances are you already have everything you need to test FIDO2.

Why FIDO2 is phishing-resistant (by design)

Traditional authentication systems rely on shared secrets, like passwords or one-time codes, that the user knows and the server also stores or verifies. This creates a fundamental weakness: if an attacker tricks the user into giving up that secret (e.g., through a phishing site), the attacker can log in as the user. Don't believe me, check out my demo with [Evilginx](#) on this very topic!



[Watch on YouTube](#)

No shared secrets = Nothing to steal

FIDO2 changes this game completely, with FIDO2:

- The private key never leaves the user's device
- The public key is stored on the server, but is useless on its own
- Authentication is bound to the website domain via the RP ID

This means even if an attacker creates a fake login page that looks like a legitimate service, the authenticator will not respond to the challenge — because the domain doesn't match the RP ID it's expecting.

Domain-bound credentials

During both registration and authentication, the authenticator validates the origin of the request using the hashed relying party ID. If the request comes from a domain it wasn't registered with (like `off1ce.com` instead of `office.com`), it refuses to complete the action.

Phishing websites can mimic the look of a real site, but they cannot fake the origin sent by the browser to the authenticator. This is enforced at the hardware level and can't be overridden by the user.

Example: why phishing fails against FIDO2

Let's say a user is tricked into visiting a phishing site:

1. The site looks like a real login page and asks for biometric authentication.
2. The user attempts to log in with their Passkey.
3. The authenticator sees that the request comes from `login.ph1sh-site.com`, not `login.microsoftonline.com`.
4. It refuses to proceed. No signature is created. Nothing is sent.

Meanwhile, the attacker gains no usable credentials, because there are none to steal.

FIDO2 isn't just harder to phish — it makes phishing fundamentally irrelevant. That's why major tech players like Apple, Google, and Microsoft are rolling out **Passkeys** as the default login method in their ecosystems and so should you!

My final thoughts

The shift from passwords to Passkeys isn't just a technological upgrade, it's a complete rethinking of how we establish digital trust. With FIDO2, we finally have a solution that aligns security with usability, and flips the script on attackers by removing the very thing they prey on: shared secrets.

“If there’s nothing to steal, nothing can be stolen.”

— *From my bachelor thesis on Passwordless Authentication, 2025*

This blog post is a direct result of the extensive research I conducted for my bachelor thesis in Cybersecurity, where I explored passwordless authentication from both a technical and human perspective. The project involved deep dives into standards like FIDO2, usability studies around Passkeys, and interviews with security professionals, all with one goal: to understand how we can replace passwords without compromising on security or user experience.

Sharing this knowledge publicly has been important to me, not just to document what I’ve learned, but to help others understand how FIDO2 works under the hood, and why it’s a real game-changer for digital identity.

As always, I hope this was informative, if you have any question, reach out to me and let me know your thoughts!